

ARRAYS 1



Good
Morning

Today's content

01. Max subarr sum
02. Zero Queries I
03. Zero Queries II
04. Rain water Trapping (Hard)

Q1. Given an arr[], find the maximum subarr sum out of all the subarrays.

arr[] = { -3 2 4 -1 3 -4 3 } Ans = 8
 0 1 2 3 4 5 6

arr[] = { -2 3 4 -1 5 -10 7 } Ans = 11

arr[] = { -3 4 6 8 -10 2 7 } Ans = 18
 0 1 2 3 4 5 6

* Brute force Approach

→ Generate all subarrays & get sum.

if (sum > overallsum) overall = sum;

No. of possible subarrays = $\frac{n(n+1)}{2}$

max = -∞

for (i = 0; i < n; i++)

 for (j = i; j < n; j++)

 sum = 0

 for (k = i; k ≤ j; k++)

 sum += A[k];

 max = maximum(max, sum);

Tc: O(n³)

Sc: O(1)

Idea 2 $\rightarrow$ Use prefix sum $\longrightarrow$ TC: $O(n^2)$
SC: $O(n)$

Idea 3 $\rightarrow$ Use carry forward technique

max = $-\infty$

for ($i=0; i < n; i++$)

sum = 0

for ($j=i; j < n; j++$) {

sum += A[j];

max = maximum(max, sum);

}

}

TC: $O(n^2)$

SC: $O(1)$

* Constraint

TC = $O(n)$

SC = $O(1)$

* Kadane's Algo

Case 1 = All elements in arr are positive

1	4	6	10	2
---	---	---	----	---

ans = sum of all elements

Case II = All elements in arr are negative

-1	-4	-6	-10	-2
----	----	----	-----	----

ans = max of array

Case III

-ve	+ve	-ve
-----	-----	-----



↳ sum of all the positive element

Case IV

+ve	+ve		
-----	-----	--	--

ans = x

y

→ add y to ans

as it is going
to increase the
subarr sum

+ve	-ve	40	
-----	-----	----	--

10

-80

Obs → if eff sum > 0, then i carryforward
the previous sum taking the contribution
of neg ele, in hope of better result

if $\text{eff sum} < 0$, then you should start
new subarr

01. $\text{arr}[] = \{ 2 \quad 3 \quad -4 \quad 6 \}$

$\text{sum} = 0 \quad 2 \quad 5 \quad 1 \quad 7$

$\text{ans} = -\infty \quad 2 \quad 5 \quad 5 \quad 7$

02. $\text{arr}[] = \{ 2 \quad 3 \quad -6 \quad 7 \}$

$\text{sum} = 0 \quad 2 \quad 5 \quad \cancel{-1} \quad 7$

$\text{ans} = -\infty \quad 2 \quad 5 \quad 5 \quad 7$

03. $\text{arr}[] = \{ -2 \quad -3 \quad -1 \quad -7 \}$

$\text{sum} = 0 \quad \cancel{-2} \quad \cancel{-3} \quad \cancel{-1} \quad \cancel{-7}$

$\text{ans} = -\infty \quad -2 \quad -2 \quad -1 \quad -1$

04. $\text{arr}[] = \{ -7 \quad 2 \quad 4 \quad 3 \quad -5 \quad 9 \}$

$\text{sum} = 0 \quad \cancel{-7} \quad 2 \quad 6 \quad 9 \quad 4 \quad 13$

$\text{ans} = -\infty \quad -7 \quad 2 \quad 6 \quad 9 \quad 9 \quad 13$

sum = 0 ans = -∞

for (i = 0; i < n; i++)

 sum += A[i];

 ans = max(ans, sum);

 if (sum < 0) {

 sum = 0;

return ans;

TC: O(n)

SC: O(1)

02. Zero queries 1

Initially all ele of an arr[] are 0 & you are given Q queries. Every query contains **idx & value**. Increment elements from ith idx to last idx by value. Return final state of arr[].

Queries

idx value

3 4

1 3

4 -2

arr[] = { 0 0 0 0 0 0 0 }

 4 4 4 4
 3 3 3 3 3 3
 -2 -2 -2

ans = 0 3 3 7 5 5 5

BF $\rightarrow$ For each query, iterate from idx to $n-1$ & update the ele of arr by value

TC: $O(Q \cdot n)$

SC: $O(1)$

* Idea 2

a_0	a_1	a_2	a_3
-------	-------	-------	-------

$$\begin{array}{cccc}
 P_f = & a_0 & a_0 & a_0 & a_0 \\
 & & + & + & + \\
 & & a_1 & a_1 & a_1 \\
 & & & + & + \\
 & & & a_2 & a_2 \\
 & & & & + \\
 & & & & a_3
 \end{array}$$

0	-3	+2	+1	0	0
0	1	2	3	4	5

Q queries

idx val

2 4

1 -3

2 -2

3 1

$P_f[] =$

0	-3	-1	0	0	0
0	1	2	3	4	5

0	1	2	3	4	5
0	0	0	0	0	0
		+4	+4	+4	+4
0	1	2	3	4	5
0	0	4	4	4	4
		-3	-3	-3	-3
0	1	2	3	4	5
0	-3	1	1	1	1
		-2	-2	-2	-2
0	1	2	3	4	5
0	-3	-1	-1	-1	-1
			1	1	1
0	1	2	3	4	5
0	-3	-1	0	0	0

Idea $\rightarrow$ Put the val at its idx, create pf array

```
for (i=0; i<Q; i++)
```

```
    idx = Queries[i][0]
```

```
    val = Queries[i][1];
```

```
    arr[idx] += val;
```

3

TC: $O(Q + N)$

SC: $O(1)$

```
int sum = 0
```

```
for (i=1; i<n; i++) {
```

```
    sum = sum + arr[i]
```

```
    arr[i] = sum;
```

3

02. Zero queries 2

Initially all ele of an arr[] are 0 & you are given Q queries. Every query contains {s, e, val}. Increment elements from s to e by value. Return final arr[]

0	1	2	3	4	5	6
0	0	0	0	0	0	0

+3 +3 +3 +3

-2 -2 -2

+4 +4 +4

Queries

st	e	val
3	6	+3
2	4	-2
1	3	+4

Ans ⇒

0	1	2	3	4	5	6
0	4	2	5	1	3	3

* Ideas

0	1	2	3	4	5	6
0	0	0	0	0	0	0

4 4 4 4 4 4

Queries

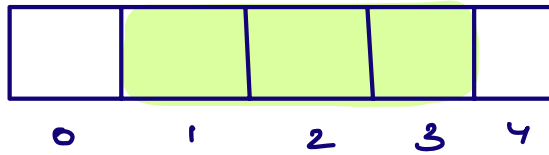
st	e	val
1	3	+4
2	4	-2
3	6	+3

st	val
1	4
2	-2
3	+3

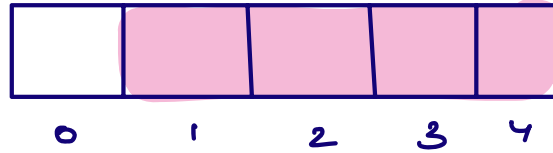
end	val
4	-4
5	+2
7	-3

→ idx is not present,
don't do anything

arr[s e]

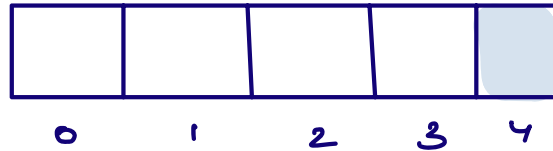


arr[s n-1] += val



arr[s] += val;

arr[e+1 n-1] -= val



arr[e+1] -= val;

int() sumqueries (int [] arr, int [] Queries)

for (i = 0; i < Q; i++) {

int s = Queries[i][0]

int e = Queries[i][1]

int val = Queries[i][2];

arr[s] += val;

if (e+1 < n) arr[e+1] -= val;

}

// create psum arr using original arr

TC: $O(Q+n)$

SC: $O(1)$

* Given an $ar[]$, Create two arrays $lmax$ & $rmax$

$lmax[i] = \text{max of all elements from } 0 \text{ to } i$

$rmax[i] = \text{max of all elements from } i+1 \text{ to } n-1$

$ar[]$	1	7	-3	4	2	-8	-6
	0	1	2	3	4	5	6

$lmax =$	1	7	7	7	7	7	7
----------	---	---	---	---	---	---	---

$rmax =$	7	7	4	4	2	-6	-6
----------	---	---	---	---	---	----	----

$lmax \rightarrow \{ \text{TODO} \}$

$rmax[n-1] = ar[n-1]$

TC: $O(n)$

SC: $O(1)$

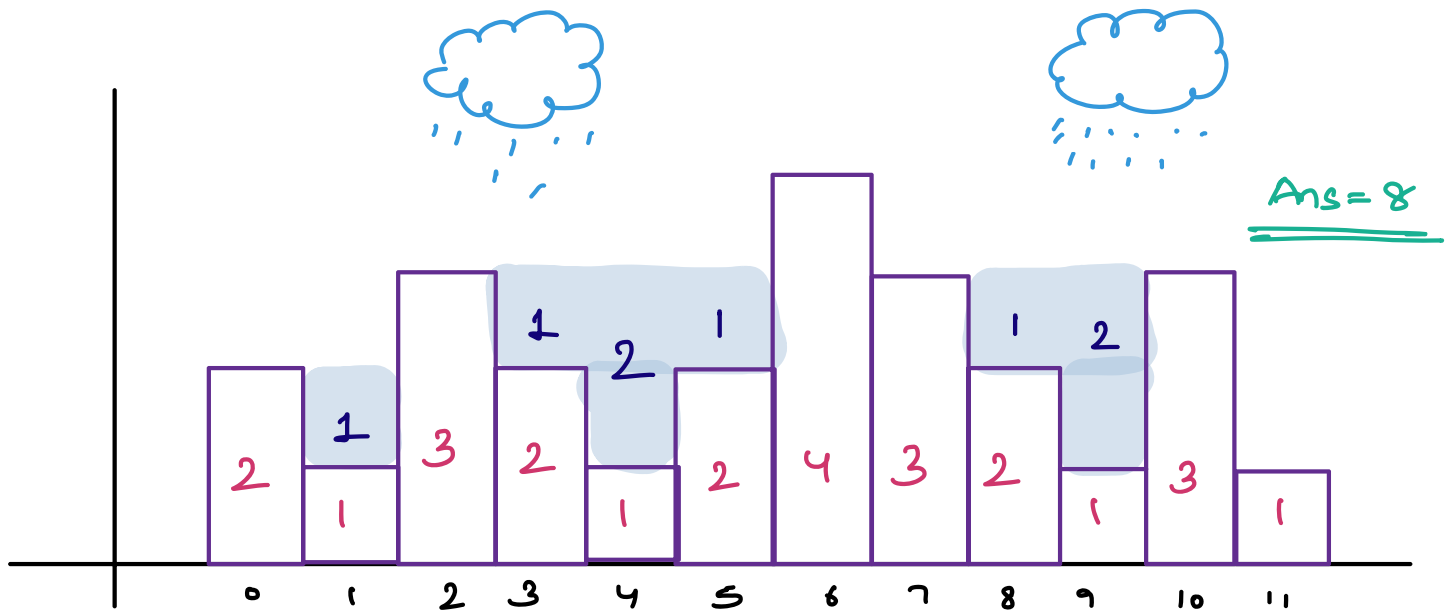
for ($i = n-2; i \geq 0; i--$)

$rmax[i] = \max(ar[i], rmax[i+1]);$

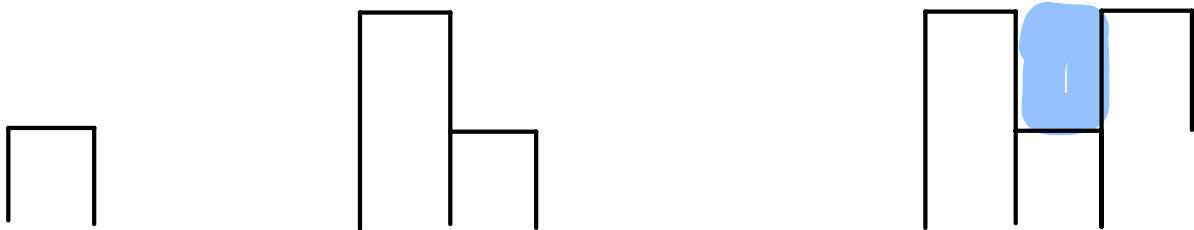
Q Rain water Trapped

Given n array elements, where $ar[i]$ represents height of the building. Return amount of water trapped on all buildings

$arr[] = \{ 2, 1, 3, 2, 1, 2, 4, 3, 2, 1, 3, 1 \}$

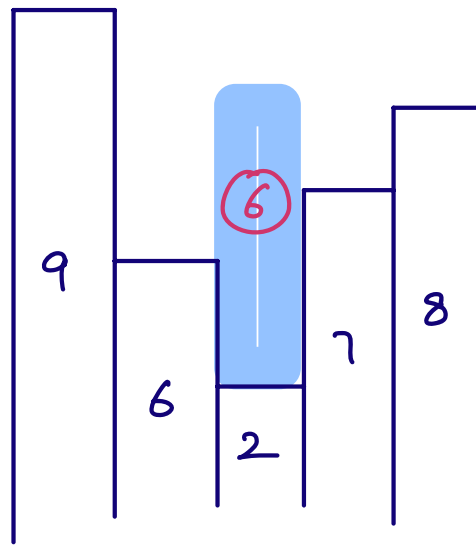


* Condition to accumulate water

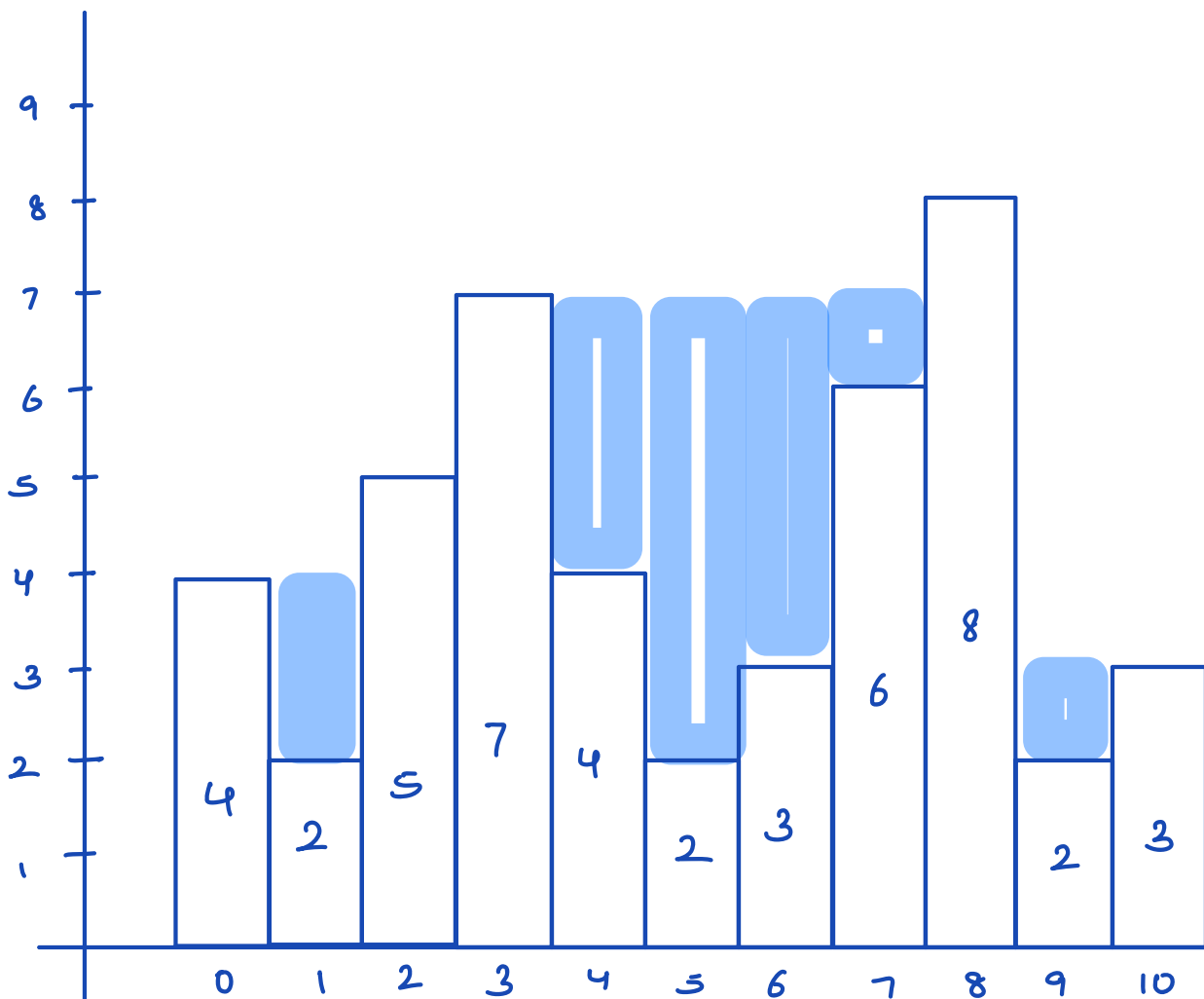


* Idea $\rightarrow$ Calculate the water accumulated at each & every building & add it.

Observation



$\min(\text{leftmax}, \text{rightmax}) - \text{ht of building}$



lmax = 4 4 5 7 7 7 7 7 8 8 8

rmax = 8 8 8 8 8 8 8 8 8 3 3

min = 4 4 5 7 7 7 7 7 8 3 3

ht = 4 2 5 7 4 2 3 6 8 2 3

$$\text{water} = 0 + 2 + 0 + 0 + 3 + 5 + 4 + 1 + 0 + 1 + 0 = \underline{\underline{16}}$$

water = 0

for (i = 1 ; i < n - 1 ; i++) {

l = iterate from 0 to i

r = iterate from i+1 to n-1

int min = minimum(l, r)

water += min - arr[i];

TC: $O(n^2)$

SC: $O(1)$

}

Idea 2 →

01. Create lmax[] & rmax[] array

water = 0

for (i = 1 ; i < n - 1 ; i++) {

l = lmax[i]

r = rmax[i]

water += min(l, r) - arr[i];

}

return water;

Tc: $O(n)$

Sc: $O(n)$

↳ $2n$

Idea 3 → Find the lmax on the go & only create rmax arr.

water = 0

l = arr[0];

for (i = 1 ; i < n - 1 ; i++) {

l = max(l, arr[i]);

r = rmax[i]

water += min(l, r) - arr[i];

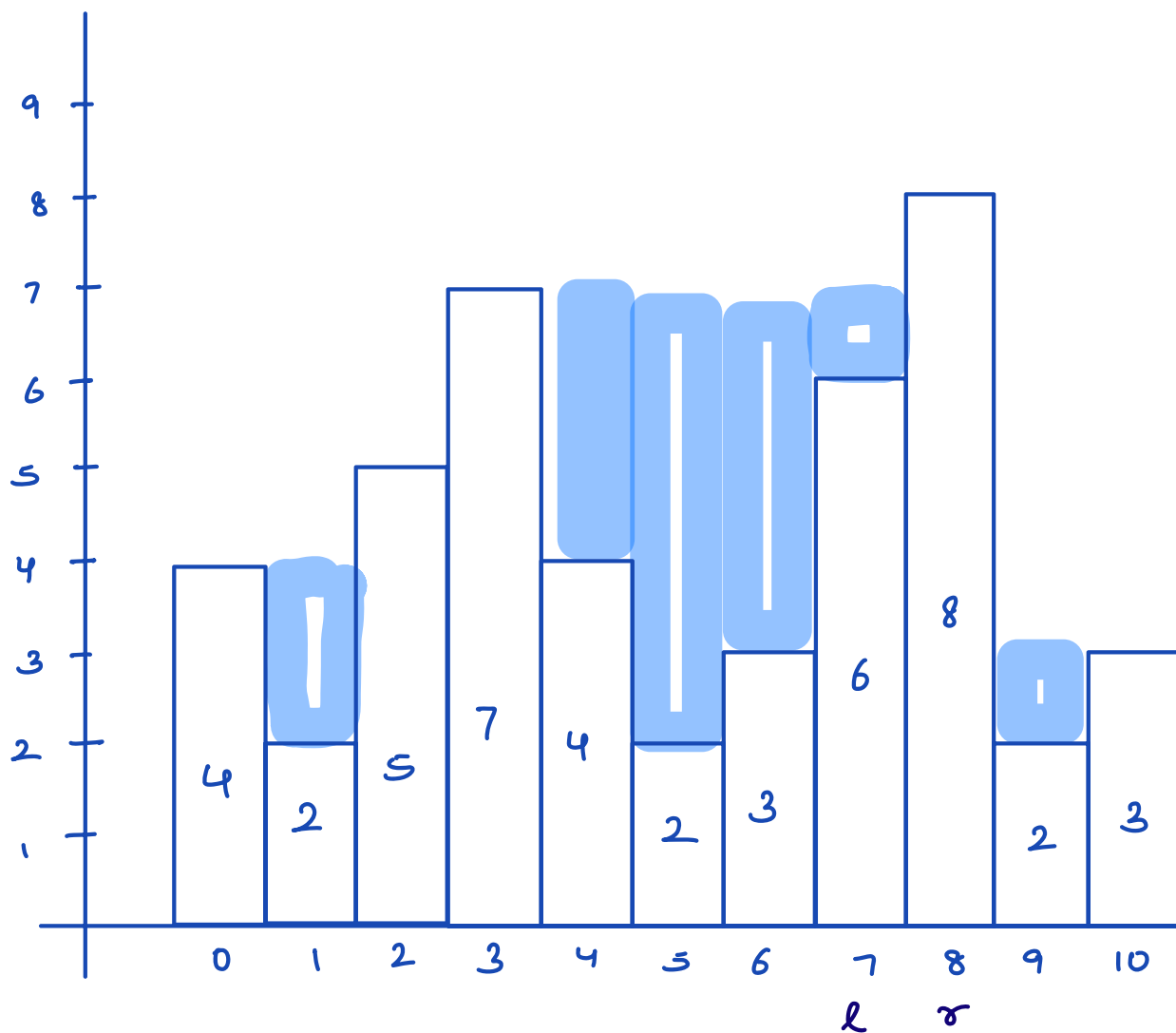
}

return water;

Tc: $O(n)$

Sc: $O(n)$

Idea 4 : Without extra space



$lmax = 4$ ~~5~~ 7

$rmax = 8$ ~~3~~

$$ans = \min(7, 8) - 4$$

$$= 7 - 4 = 3$$

lmax = 0

rmax = 0

i = 0 , j = n-1

while (i < j) {

lmax = Math.max(lmax, ar[i])

rmax = Math.max(rmax, ar[j])

if (lmax < rmax) {

ans += lmax - ar[i];

i++;

2

else {

ans += rmax - ar[j];

j--;

3

3

Tc: $O(n)$

Sc: $O(1)$